In [1]:
```
import os
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

path = "titanic_train.csv"
df = pd.read_csv(path)
print(df.shape)
print("Missing ages:", df["Age"].isna().sum())
```

```
(891, 12)
Missing ages: 177
```

for the missing ages I will use median because it is more skew resistant than mean.

In [2]:
```
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
features = ["Pclass", "Sex", "Age", "Fare"]
X = df[features].copy()
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

age_median = X_train["Age"].median()
fare_median = X_train["Fare"].median()
for part in (X_train, X_test):
    part["Age"] = part["Age"].fillna(age_median)
    part["Fare"] = part["Fare"].fillna(fare_median)

print("Train median age used:", age_median)
print(X_train.shape, X_test.shape)
```

```
Train median age used: 28.0
(623, 4) (268, 4)
```

In [3]:
```
models = {
    "Decision tree (max_depth=3)": DecisionTreeClassifier(max_depth=3, randc
    "kNN (k=7, scaled)": make_pipeline(StandardScaler(), KNeighborsClassifie
}
fitted = {}
for name, m in models.items():
    m.fit(X_train, y_train)
    fitted[name] = m
    print(f"{name}: train acc={m.score(X_train, y_train):.3f}  TEST acc={m.s
```

```
Decision tree (max_depth=3): train acc=0.827  TEST acc=0.806
kNN (k=7, scaled): train acc=0.849  TEST acc=0.799
```

The decision tree(max_depth=3) is what I would go with because the accuracy difference between the models(tree 0.806, kNN 0.799) is noise and it is simpler.

In [4]:
```
tree = fitted["Decision tree (max_depth=3)"]
res = X_test.copy()
res["Survived"] = y_test
res["Pred"] = tree.predict(X_test)
res["Wrong"] = res["Survived"] != res["Pred"]

print("By actual outcome:")
display(res.groupby("Survived")["Wrong"].agg(["mean", "sum", "count"]).round
print("By sex (0=male, 1=female) x class:")
display(res.groupby(["Sex", "Pclass"])["Wrong"].agg(["mean", "sum", "count"]
```

```
By actual outcome:
```

Survived	mean	sum	count
0	0.108	17	157
1	0.315	35	111

```
By sex (0=male, 1=female) x class:
```

Sex	Pclass	mean	sum	count
0	1	0.455	20	44
	2	0.080	2	25
	3	0.122	12	98
1	1	0.000	0	33
	2	0.185	5	27
	3	0.317	13	41

In [5]:
```
fc_men = res[(res["Sex"] == 0) & (res["Pclass"] == 1)]
print(fc_men.groupby(["Survived", "Pred"]).size())
```

```
Survived  Pred
0         0       24
1         0       20
dtype: int64
```

The passengers that the model got wrong most frequently were first-class men, where it wrongly predicts death almost half of the time.